Software Engineering Lab

Lab 23

Optimistic vs Pessimistic Locking & Atomic Updates

Mekanisme Pencegahan Lost Update, Version Guard, dan Operasi Atomik di Go



Lukman / lukman-ss

Anomali Lost Update: Korupsi Data Tanpa Sinyal Error

Dua transaksi membaca snapshot identik, menghitung mandiri, lalu saling menimpa data.

DAMPAK LOST UPDATE PADA SISTEM PRODUKSI

Penulisan Siluman

Transaksi kedua menimpa mutasi transaksi pertama tanpa membangkitkan galat atau exception pada aplikasi.

DAMPAK LOST UPDATE PADA SISTEM PRODUKSI

Invarian Bisnis Hancur

Saldo dompet atau stok barang menyimpang dari rumus matematis: stock != initial - total_deductions.

DAMPAK LOST UPDATE PADA SISTEM PRODUKSI

Bukan Berarti Aman

Membungkus query dengan blok transaksi SQL biasa bukan berarti kebal terhadap tabrakan read-modify-write.

Ketika dua request bersamaan membaca stok bernilai 100 lalu menguranginya sebesar 1 unit, keduanya menuliskan angka 99. Pembaruan pertama hilang tanpa jejak dan tanpa menghasilkan pesan error apapun di database. Membungkus query ke dalam transaksi SQL bukan berarti data otomatis aman dari korupsi ini.

Independent read-modify-write cycles over shared state silently obliterate intermediate committed modifications.

Mengapa Isolasi Default Database Gagal Mencegah Lost Update

Tingkat isolasi standar READ COMMITTED membiarkan pembaruan data saling menimpa.

KELEMAHAN ISOLASI STANDAR PADA READ-MODIFY-WRITE

READ COMMITTED Default

PostgreSQL dan Oracle memakai READ COMMITTED; setiap query membaca snapshot committed terbaru.

KELEMAHAN ISOLASI STANDAR PADA READ-MODIFY-WRITE

Celah Pemisahan Query

Jeda waktu antara SELECT dan UPDATE memberi ruang bagi transaksi lain menyisipkan commit baru.

KELEMAHAN ISOLASI STANDAR PADA READ-MODIFY-WRITE

Tidak Otomatis Aman

Isolasi database tidak otomatis memvalidasi apakah baris yang di-update telah berubah sejak dibaca.

Review kode manual kerap terkecoh karena mengira blok transaksi ACID sudah cukup melindungi integritas mutasi. Tingkat isolasi READ COMMITTED tidak otomatis mencegah lost update pada alur read-modify-write terpisah, karena setiap query membaca snapshot committed terbaru tanpa memblokir writer lain.

Default isolation levels isolate individual queries, not multi-statement application calculation lifecycles.

Anatomi Tabrakan Waktu: Dua Transaksi Berebut Stok

Kronologi urutan eksekusi yang melenyapkan mutasi pertama di penyimpanan.

SNAPSHOT T1 & T2

1. Baca Serentak

T1 dan T2 membaca stok awal 100 secara bersamaan sebelum ada komit baru.

STATE DI MEMORI

2. Kalkulasi Terpisah

T1 menghitung 100 - 1 = 99; T2 juga menghitung 100 - 1 = 99 di memori terpisah.

WRITE DATABASE

3. Komit T1

T1 menyimpan stok = 99 ke database; state berubah tanpa diketahui proses T2.

SILENT OVERWRITE

4. Komit T2

Hanya pada jalur ini T2 menimpa data menjadi 99, melenyapkan hasil mutasi T1.

Kronologi ini memperlihatkan celah fatal ketika dua proses membaca angka awal yang sama lalu melakukan kalkulasi di memori aplikasi. Transaksi kedua menimpa pembaruan transaksi pertama tanpa menyadari versinya sudah usang. Hanya pada jalur ini mutasi ganda tereduksi menjadi pengurangan tunggal.

Unsynchronized read-then-write pipelines collapse concurrent increments into stale, overwritten states.

Tiga Taksonomi Strategi Pengendalian Konkurensi

Membandingkan alur kerja Pessimistic Locking, Optimistic Locking, dan Atomic Statement.

SELECT FOR UPDATE

1. Pessimistic Lock

Kunci baris saat dibaca; transaksi konkuren antre di lock hingga commit selesai.

VERSION COLUMN

2. Optimistic Guard

Baca tanpa lock; saat update cek version = V. Gagal jika affected_rows == 0.

SINGLE STATEMENT

3. Atomic Update

Pangkas read-modify-write: UPDATE ... SET stock=stock-qty WHERE stock>=qty.

SIMULASI IN-MEMORY

4. Evaluasi Lab

Lab membuktikan logika ketiganya; belum membuktikan engine lock manager MySQL riil.

Arsitektur modern menawarkan tiga strategi untuk menangkal lost update: memblokir baris dengan pessimistic lock, mendeteksi konflik pada commit dengan optimistic guard, atau memadatkan mutasi ke satu query atomik. Pengujian lab membuktikan validitas ketiga logika, tetapi belum membuktikan engine lock table PostgreSQL/MySQL berkapasitas jutaan baris.

Select concurrency boundaries: block access proactively, validate versions defensively, or execute mutations atomically.

Pola Naif Read-Modify-Write yang Membuka Celah Tabrakan

Pemisahan fase baca dan tulis memicu anomali penulisan yang hilang di Go.

```
func (s *Store) NaiveDeduct(id int, qty int) error {
    p, err := s.Get(id) // 1. Read snapshot tanpa lock
    if err != nil { return err }
    if p.Stock < qty { return ErrInsufficientStock }
    time.Sleep(100 * time.Microsecond) // perlebar celah balapan
    s.mu.Lock()
    curr := s.products[id]
    curr.Stock = p.Stock - qty // 2. Tulis memakai snapshot usang
    s.mu.Unlock()
    atomic.AddInt64(&s.NaivelyDrawn, int64(qty))
    return nil
}
```

Fungsi NaiveDeduct membaca stok tanpa mengunci baris, lalu melakukan kalkulasi sebelum menulis kembali. Menyisipkan jeda microsecond bukan berarti membuat sistem sengaja cacat, melainkan merefleksikan latensi I/O aplikasi yang nyata di lingkungan produksi berbeban tinggi.

Separating state inspection from state mutation without synchronization invites concurrency collision.

Kronologi Kegagalan: 50 Goroutine dan Hilangnya 49 Mutasi

Bagaimana 50 pengurangan stok berakhir dengan sisa stok 99 alih-alih 50.

ALUR PENURUNAN INTEGRITAS INVENTARIS

Fase 1: Ledakan Akses

50 goroutine membaca stok awal 100 secara paralel saat proses sistem baru dimulai.

ALUR PENURUNAN INTEGRITAS INVENTARIS

Fase 2: Perhitungan Identik

Seluruh goroutine menghitung nilai baru 100 - 1 = 99 karena memegang data lokal yang sama.

ALUR PENURUNAN INTEGRITAS INVENTARIS

Fase 3: Timpa Bertubi-tubi

Satu per satu goroutine menulis nilai 99 ke penyimpanan memori global secara bergantian.

ALUR PENURUNAN INTEGRITAS INVENTARIS

Fase 4: Anomali Stok 99

Hanya pada jalur ini 50 mutasi berhasil tercatat namun stok fisik gudang hanya berkurang 1 unit.

Saat 50 goroutine memanggil NaiveDeduct serempak pada stok awal 100, seluruh pekerja membaca angka 100 dan masing-masing menulis 99. Hanya pada jalur ini terjadi bencana inventaris riil: log transaksi mencatat 50 barang terjual, namun database mencatat stok tersisa 99.

Concurrent stale overwrites cause severe inventory inflation and unrecoverable ledger drift.

Matriks Strategi: Evaluasi Tiga Pendekatan Mutasi Konkuren

Membedakan karakteristik teknis, tingkat konkurensi, dan risiko operasional.

Dimensi Evaluasi	Naive Read-Modify-Write	Pessimistic Locking	Optimistic Locking	Atomic Update
Mekanisme Kunci	Tanpa isolasi mutasi	Row lock eksklusif	Version check saat commit	Single-statement lockless
Performa Beban Baca	Sangat cepat (tanpa lock)	Cepat (MVCC read aman)	Sangat cepat (tanpa lock)	Sangat cepat
Tingkat Konflik Tinggi	Korupsi data massal	Terblokir antrean (antre)	Lonjakan retry / error 409	Optimal (sekuensial cepat)
Risiko Sistem	Lost update tanpa error	Deadlock / antrean panjang	Starvation pada retry loop	Terbatas pada aritmatika simpel
Skenario Ideal	Hanya sistem single-	Saldo bank / tiket terbatas	Edit profil / dokumen CMS	Counter hit / decrement stok

Tabel ini memetakan trade-off komprehensif antara pendekatan naif, pessimistic locking, optimistic locking, dan pembaruan atomik. Pessimistic locking cocok untuk data bernilai tinggi dengan kontensi padat, sedangkan optimistic locking unggul pada beban baca dominan yang jarang mengalami tabrakan.

Match concurrency control primitives directly to data criticality, conflict probability, and update complexity.

Alur Pengambilan Keputusan Strategi Penguncian Data

Tiga kriteria teknis menentukan pemilihan strategi konkurensi yang tepat sasaran.

PANDUAN PEMILIHAN STRATEGI MUTASI

1. Operasi Tunggal?

Jika mutasi hanya melibatkan pengurangan numerik sederhana tanpa aturan silang tabel, gunakan Atomic Update.

PANDUAN PEMILIHAN STRATEGI MUTASI

2. Kontensi Padat?

Jika entitas sering diperebutkan (flash sale, saldo), gunakan Pessimistic Lock (SELECT FOR UPDATE).

PANDUAN PEMILIHAN STRATEGI MUTASI

3. Baca-Dominan?

Bila konflik jarang, gunakan Optimistic Version Guard; bukan berarti tanpa retry, siapkan backoff.

Keputusan arsitektur bertumpu pada karakteristik mutasi data. Jika mutasi adalah pengurangan sederhana satu baris, atomic update adalah pilihan paling efisien. Memilih optimistic locking bukan berarti bebas dari penanganan kegagalan; lapisan pemanggil wajib menyiapkan algoritma retry backoff.

Navigate concurrency solutions by evaluating mutation locality, contention intensity, and operational retry costs.

Bukti Eksekusi: Verifikasi Lima Skenario dan Race Detector

Output terminal membuktikan anomali stok naif dan keberhasilan ketiga proteksi.

```
=== RUN   TestNaiveLostUpdate
    locking_test.go:37: Lost Update Demonstrated: final stock is 99 (expected 50)
--- PASS: TestNaiveLostUpdate (0.00s)
=== RUN   TestPessimisticLocking -> PASS: (stock == 50)
=== RUN   TestOptimisticLockingConflict -> PASS: (1 succeeded, 19 conflicts rejected)
=== RUN   TestOptimisticLockingWithRetry -> PASS: (20 total updates applied, stock == 80)
=== RUN   TestAtomicConditionalUpdate -> PASS: (stock == 50)
PASS: ok  labs/23-optimistic-vs-pessimistic-locking/tests  2.510s (race detector: clean)
[1] Naive Read-Modify-Write: Stock=99 (LOST UPDATE DETECTED!)
[2] Pessimistic Locking:    Stock=50 (SUCCESS - Fully Synchronized)
[3] Optimistic Direct:      Stock=99 (SUCCESS - 1 Succeeded, 19 Conflicts Rejected)
[4] Optimistic With Retry:  Stock=80 (SUCCESS - 20 Succeeded, 61 Retries Absorbed)
[5] Atomic Update:          Stock=50 (SUCCESS - Lockless Single Statement)
```

Hasil eksekusi test suite dan demo CLI mengonfirmasi keakuratan seluruh strategi pengujian. Race detector Go melaporkan status bersih tanpa data race pada memori, sementara pemeriksaan logika membuktikan bahwa pendekatan naif melenyapkan 49 mutasi stok.

Runtime test execution proves memory safety while domain assertions verify transactional business invariants.

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan batasan simulasi Go in-memory terhadap kompleksitas RDBMS produksi.

BATAS PEMBUKTIAN SISTEM LAB 23

YANG DIBUKTIKAN

Reproduksi anomali lost update, proteksi row-lock per item, penolakan version mismatch, dan konvergensi retry.

BATAS PEMBUKTIAN SISTEM LAB 23

YANG TIDAK DIBUKTIKAN

Engine lock table database riil, deadlock detector lintas-tabel, gap locking, dan isolasi MVCC fisik RDBMS.

BATAS PEMBUKTIAN SISTEM LAB 23

BATAS KLAIM EMPIRIS

Lab belum membuktikan throughput performa riil di bawah beban 100.000 transaksi terdistribusi.

Lab ini menggunakan simulasi sync.Mutex in-memory untuk memodelkan perilaku penguncian tingkat baris dan version check. Implementasi ini belum membuktikan mekanisme kompleks RDBMS fisik seperti lock manager tabel, gap locks, atau deadlock detection otomatis pada transaksi multi-baris.

Carefully demarcate application-level concurrency simulations from enterprise storage engine lock internals.

Kompromi Rekayasa: Beban Tunggu vs Beban Percobaan Ulang

Mengevaluasi konsekuensi throughput, latensi, dan kompleksitas kode aplikasi.

ANALISIS TRADE-OFF PENGUNCIAN SISTEM

Pessimistic: Latensi Antre

Jaminan konsistensi tinggi tanpa retry, namun latensi naik drastis saat baris yang sama diserbu transaksi.

ANALISIS TRADE-OFF PENGUNCIAN SISTEM

Optimistic: Beban CPU/Retry

Throughput baca maksimal tanpa blocking, namun memboroskan komputasi saat conflict rate meninggi.

ANALISIS TRADE-OFF PENGUNCIAN SISTEM

Tidak Otomatis Efisien

Mengandalkan retry tidak otomatis efisien bila kontensi tinggi memicu fenomena starvation.

Setiap strategi menuntut ongkos operasional yang berbeda. Pessimistic locking mengorbankan waktu tunggu antrean transaksi, sedangkan optimistic locking mengonsumsi siklus CPU dan bandwidth jaringan untuk mencoba ulang transaksi yang gagal. Menggunakan retry tidak otomatis efisien jika konflik terjadi terus-menerus.

Balance thread queue serialization delays against compute and network retry amplification costs.

Simulasi Row Lock Eksklusif Berbasis Mutex Per Baris

Memodelkan semantik SELECT FOR UPDATE di memori dengan row-level lock map.

```
type Store struct {
    mu       sync.Mutex          // melindungi struktur map internal
    rowLocks map[int]*sync.Mutex // lock eksklusif per ID produk
    products map[int]*Product    // data penyimpanan stok dan versi
}
func (s *Store) GetRowLock(id int) *sync.Mutex {
    s.mu.Lock()
    defer s.mu.Unlock()
    lock, exists := s.rowLocks[id]
    if !exists { lock = &sync.Mutex{}; s.rowLocks[id] = lock }
    return lock
}
```

Store mengisolasi penguncian dengan memisahkan mutex struktur peta dari mutex baris individu. Simulasi ini tidak sama dengan lock manager PostgreSQL yang mengelola shared lock, exclusive lock, dan lock timeout, namun secara tepat merepresentasikan sifat serialisasi akses baris tunggal.

Decouple structural table mutexes from granular entity locks to model row-level database serialization.

internal/inventory/store.go:9–52

Transformasi: Dari Overwrite Diam-diam ke State Terproteksi

Pergeseran arsitektur mutasi data dari pembaruan tanpa pelindung ke kontrol ketat.

KOMPARASI STATUS EKSEKUSI MUTASI

Sebelum: Naive RMW

50 pemanggilan fungsi mutasi; stok akhir terkorupsi di angka 99; hilangnya 49 mutasi tanpa jejak peringatan.

KOMPARASI STATUS EKSEKUSI MUTASI

Sesudah: Protected Invariant

Seluruh mutasi terkontrol; stok akhir tepat 50 (pessimistic/atomic) atau 80 dengan retry konvergen.

KOMPARASI STATUS EKSEKUSI MUTASI

Bukan Berarti Rumit

Menjaga integritas bukan berarti harus rumit; atomic update menyelesaikan masalah dalam satu query SQL.

Sebelum penerapan strategi konkurensi, data inventaris hancur secara diam-diam akibat mutasi yang saling menimpa. Setelah proteksi diterapkan, invarian matematis stok terjaga sempurna di bawah beban serempak. Menerapkan proteksi bukan berarti menambah beban kode rumit jika kebutuhan mutasi dapat diselesaikan via query atomik.

Shift from blind data overwrites to guarded state invariants with minimal architectural overhead.

Pessimistic Locking: Mengunci Sebelum Membaca Data

Mengamankan siklus read-modify-write dengan menahan lock eksklusif hingga tuntas.

```
func (s *Store) PessimisticDeduct(id int, qty int) error {
    rowLock := s.GetRowLock(id)
    rowLock.Lock() // akuisisi lock eksklusif baris
    defer rowLock.Unlock()
    s.mu.Lock()
    p, exists := s.products[id]
    if !exists { s.mu.Unlock(); return ErrNotFound }
    if p.Stock < qty { s.mu.Unlock(); return ErrInsufficientStock }
    p.Stock -= qty
    s.mu.Unlock()
    atomic.AddInt64(&s.Pessimistically, int64(qty))
    return nil
}
```

Fungsi PessimisticDeduct mengunci rowLock sebelum melakukan pengecekan stok dan mutasi nilai. Logika pencegahan ini cakupannya mengikuti test konkurensi lab, membuktikan bahwa seluruh goroutine mengantre secara tertib sehingga stok berkurang tepat 50 unit tanpa ada mutasi yang hilang.

Enforce exclusive ownership of mutable state throughout the entire read-compute-write transaction lifecycle.

internal/inventory/store.go:93-116

Optimistic Locking: Version Guard Tanpa Memblokir Pembaca

Mendeteksi konflik pada fase tulis dengan memvalidasi versi data snapshot.

```
func (s *Store) OptimisticDeduct(id int, qty int) error {
    p, err := s.Get(id) // ambil snapshot beserta nilai version
    if err != nil { return err }
    if p.Stock < qty { return ErrInsufficientStock }
    time.Sleep(50 * time.Microsecond)
    s.mu.Lock()
    defer s.mu.Unlock()
    curr, exists := s.products[id]
    if curr.Version != p.Version { // cek version guard
        atomic.AddInt64(&s.OptimisticFails, 1)
        return ErrOptimisticLock // equivalen affected_rows == 0
    }
    curr.Stock -= qty; curr.Version++
    atomic.AddInt64(&s.Optimistically, int64(qty)); return nil
}
```

Optimistic locking membaca data beserta nomor versinya tanpa menahan kunci baris. Saat commit, versi divalidasi; jika versi di memori telah berubah, update digagalkan dengan mengembalikan ErrOptimisticLock. Strategi ini tidak otomatis sukses jika aplikasi tidak menangani error konflik tersebut.

Validate state immutability at commit time; reject modifications whenever baseline versions diverge.

internal/inventory/store.go:120–153

Konvergensi Optimistik: Exponential Backoff dengan Jitter

Meredam benturan thundering-herd saat mencoba ulang mutasi yang gagal.

```
func (svc *Service) DeductOptimisticWithRetry(id, qty, maxRetries int) error {
    for attempt := 0; attempt <= maxRetries; attempt++ {
        err := svc.store.OptimisticDeduct(id, qty)
        if err == nil { return nil }
        if err != ErrOptimisticLock { return err }
        if attempt == maxRetries { return ErrOptimisticLock }
        sleepDuration := time.Duration(1<
```

Ketika konflik versi terjadi, Service melakukan percobaan ulang dengan jeda bertingkat eksponensial ditambah angka acak jitter. Jitter mencegah goroutine mencoba kembali pada detik yang sama persis. Hanya pada jalur ini 20 request konkuren yang saling berebut akhirnya konvergen dan sukses seluruhnya.

De-synchronize competing retries via exponential randomized pauses to guarantee system-wide convergence.

Atomic Update: Memangkas Celah Read–Modify–Write

Melakukan pengecekan kondisi dan pembaruan data dalam satu siklus atomik.

```
// Ekuivalen SQL: UPDATE products SET stock = stock - ? WHERE id = ? AND stock >= ?
func (s *Store) AtomicDeduct(id int, qty int) error {
    if qty <= 0 { return ErrInvalidQuantity }
    s.mu.Lock()
    defer s.mu.Unlock()
    curr, exists := s.products[id]
    if !exists { return ErrNotFound }
    if curr.Stock < qty { return ErrInsufficientStock }
    curr.Stock -= qty // validasi dan kalkulasi dalam satu seksi kritis
    atomic.AddInt64(&s.Atomically, int64(qty))
    return nil
}
```

AtomicDeduct menghilangkan jeda waktu antara membaca dan menulis dengan menggabungkan validasi stok dan pengurangan nilai ke dalam satu seksi kritis. Pola ini bukan berarti solusi untuk semua masalah, karena operasi atomik satu statement tidak mampu menangani validasi multi-tabel yang kompleks.

Condense condition validation and state mutation into a unified, non-preemptible database operation.

Verifikasi Invarian: Konsistensi Matematis di Bawah Beban

Memastikan rumus InitialStock − TotalSuccessfulDeductions terpenuhi mutlak.

```
func TestOptimisticLockingConflict(t *testing.T) {
    // 20 goroutines serempak DeductOptimisticDirect pada stok 100
    wg.Wait()
    p, _ := store.Get(3)
    // Pembuktian invarian: transaksi sukses + sisa stok wajib tepat 100
    if successCount+int64(p.Stock) != 100 {
        t.Fatalf("invarian stok rusak: sukses %d + stok %d != 100",
                 successCount, p.Stock)
    }
    if conflictCount == 0 { t.Fatalf("harus ada minimal 1 konflik") }
}
```

Test suite memverifikasi bahwa penolakan optimistik tidak merusak integritas sistem: jumlah request yang sukses ditambah sisa stok wajib tepat bernilai 100. Verifikasi empiris ini cakupannya mengikuti test suite otomatis lab, membuktikan nol korupsi data meski terjadi penolakan massal.

Validate system correctness by proving conservation invariants across concurrent execution boundaries.

Checklist Penerapan Concurrency Control di Produksi

Pedoman teknis memilih dan mengonfigurasi mekanisme penguncian database.

- ✓ Identifikasi alur mutasi: apakah single-statement atomik atau read-modify-write multi-langkah.
- ✓ Gunakan UPDATE ... WHERE stock >= N untuk pembaruan counter/inventaris numerik sederhana.
- ✓ Terapkan SELECT ... FOR UPDATE pada transaksi bernilai tinggi dengan tingkat tabrakan tinggi.
- ✓ Jangan pernah menahan row lock database saat melakukan panggilan jaringan eksternal (HTTP/RPC).
- ✓ Tambahkan kolom version integer/timestamp dan periksa affected_rows == 0 untuk optimistic lock.
- ✓ Sediakan wrapper retry dengan randomized jittered backoff untuk menangani konflik optimistik.
- ✓ Jalankan race detector Go (go test -race ./...) secara berkala pada seluruh pipeline CI/CD.

Enforce systematic production checklists to prevent concurrency bugs and runtime transaction starvation.

Anti-Pattern Umum dalam Penanganan Transaksi Konkuren

Kesalahan desain arsitektur yang sering merusak konsistensi data di database.

KESALAHAN DESAIN MUTASI YANG HARUS DIHINDARI

Mengabaikan Affected Rows

Tidak memeriksa jumlah baris terpengaruh pada query update optimistik, menganggap eksekusi selalu berhasil.

KESALAHAN DESAIN MUTASI YANG HARUS DIHINDARI

Lock Menyeberang I/O

Memegang lock database saat menunggu respons webhook payment gateway; memicu antrean sistemik.

KESALAHAN DESAIN MUTASI YANG HARUS DIHINDARI

Bukan Berarti Perlu Redis

Langsung memakai Redis lock bukan berarti tepat; database relasional sudah punya locking bawaan.

Kesalahan paling umum di industri adalah menganggap query UPDATE selalu berhasil tanpa mengecek affected_rows, serta menahan lock database sambil menunggu panggilan HTTP pihak ketiga. Memasang distributed lock seperti Redis bukan berarti selalu tepat, karena database lokal sudah menyediakan mekanisme locking bawaan yang jauh lebih efisien.

Eliminate cross-network lock retention and utilize native database primitives before introducing external lock managers.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Verifikasi mandiri anomali lost update dan ketiga strateginya tanpa dependensi.

```
cd labs/23-optimistic-vs-pessimistic-locking
go test -v ./...    # verifikasi 6 test skenario naive, pessimistic, optimistic & atomic
go test -race ./... # pembuktian sinkronisasi konkuren bebas data race di Go memory
go run ./cmd/demo   # eksekusi demonstrasi CLI interaktif kelima skenario berdampingan
```

Seluruh pengujian dibangun murni menggunakan pustaka standar Go tanpa modul pihak ketiga. Eksperimen lokal ini membuktikan logika locking di memori secara cepat, namun production masih perlu mengonfigurasi batas lock timeout dan query analyzer pada database relasional yang sebenarnya.

Zero-dependency concurrency harnesses provide rapid, deterministic validation of distributed transactional patterns.

Prinsip Inti: Tidak Ada Solusi Tunggal untuk Konkurensi

Keseimbangan antara pencegahan proaktif, deteksi optimistik, dan kesederhanaan.

TIGA PILAR PENGENDALIAN MUTASI DATA

Pessimistic: Data Kritis

Pilihan utama bila tabrakan data berbiaya tinggi dan pembatalan transaksi tidak dapat ditoleransi pengguna.

TIGA PILAR PENGENDALIAN MUTASI DATA

Optimistic: Skalabilitas Baca

Pilihan optimal bila sistem dominan membaca data dan volume konflik tulis berada di tingkat rendah.

TIGA PILAR PENGENDALIAN MUTASI DATA

Atomic: Desain Sederhana

Pilihan terbaik untuk mutasi numerik tunggal; tidak otomatis rumit bila bisa diselesaikan satu query.

Kunci arsitektur transaksional yang matang adalah menyesuaikan alat dengan karakteristik kontensi data. Transaksi kritis membutuhkan kepastian pemblokiran pessimistic, aplikasi baca-banyak memanfaatkan skalabilitas optimistic, dan counter sederhana diselesaikan oleh query atomik. Merancang sistem transaksional tidak otomatis membutuhkan solusi rumit bila kebutuhan mutasi dapat diakomodasi oleh operasi atomik database.

Architectural excellence lies in pairing contention patterns with the exact concurrency primitive required.

Lost Update adalah korupsi data diam-diam. Transaksi biasa tidak mencegahnya; desain penguncian yang tepat yang menyelesaikannya.

Lost updates are silent transactional hazards eradicated only through deliberate, explicit concurrency controls.

Software Engineering Lab 23 – Optimistic vs Pessimistic Locking & Atomic Updates

internal/inventory/model.go • internal/inventory/store.go
internal/inventory/service.go • cmd/demo/main.go
tests/locking_test.go • content/02-master-draft.md

REPOSITORY

github.com/lukman-ss/software-engineering-lab/tree/main/labs/23-optimistic-vs-pessimistic-locking

Lukman / lukman-ss

Commit: aad5d78 • Status: APPROVED
Test passing: 6 automated test scenarios (LostUpdate, Pessimistic, InsufficientStock, OptimisticConflict, OptimisticRetry, AtomicUpdate)
Race detector: Clean (go test -race ./... PASS in 2.510s)
Zero external dependencies: 100% Go standard library.